CROP DISEASE DETECTION APP

Detailed Build Prompt Library

PHASE 2 — Backend & Database (FastAPI)

Prompts 56–110 · 55 Fully Detailed Prompts

Prepared for: Priyanshu Kumar Gupta & Team
(Sadhana Bag, Kumar Gaurav, Rajdeep Kumar)

Part 2 of 4 — execute this phase fully before opening the next

Phase Overview

This phase builds the authenticated API that wraps the frozen model from Phase 1 and gives every user their own private account, history, and stats.

Reference: Full Project File Structure

Every prompt in this document references paths from the following master file tree. Keep it visible while executing.

```
crop-disease-detection/
|-- README.md
|-- ml/
|   |-- requirements.txt
|   |-- dataset/
|   |   |-- raw/
|   |   |-- filtered/
|   |   |-- split/
|   |   |   |-- train/<class>/
|   |   |   |-- val/<class>/
|   |   |   `-- test/<class>/
|   |   `-- real_world_sample/<class>/
|   |-- notebooks/
|   |   |-- 00_colab_setup.ipynb
|   |   |-- 01_dataset_exploration.ipynb
|   |   `-- 02_preprocessing_test.ipynb
|   |-- preprocessing/
|   |   |-- loader.py
|   |   |-- augmentation.py
|   |   |-- dataset_pipeline.py
|   |   `-- class_weights.py
|   |-- training/
|   |   |-- baseline_cnn.py
|   |   |-- train_baseline.py
|   |   |-- transfer_mobilenetv2.py
|   |   |-- train_transfer.py
|   |   `-- transfer_resnet50.py
|   |-- evaluation/
|   |   |-- evaluate.py
|   |   |-- confusion_matrix.py
|   |   |-- compare_models.py
|   |   |-- confidence_threshold_experiment.py
|   |   `-- real_world_eval.py
```

```
|   `-- models/
|       |-- crop_disease_model.keras
|       `-- class_names.json
|
|-- backend/
|   |-- requirements.txt
|   |-- .env.example
|   |-- alembic/
|   |-- scripts/
|   |   `-- reset_db.py
|   |-- tests/
|   |   |-- conftest.py
|   |   |-- test_auth.py
|   |   |-- test_users.py
|   |   |-- test_predict.py
|   |   `-- test_history_feedback.py
|   `-- app/
|       |-- main.py
|       |-- core/
|       |   |-- config.py
|       |   |-- security.py
|       |   `-- dependencies.py
|       |-- db/
|       |   |-- session.py
|       |   `-- seed_diseases.py
|       |-- models/
|       |   |-- user.py
|       |   |-- prediction.py
|       |   |-- disease.py
|       |   `-- feedback.py
|       |-- schemas/
|       |   |-- auth.py
|       |   |-- user.py
|       |   `-- prediction.py
|       |-- routes/
|       |   |-- auth.py
|       |   |-- users.py
|       |   |-- predict.py
|       |   `-- diseases.py
|       |-- services/
|       |   |-- model_service.py
|       |   `-- preprocessing_service.py
|       `-- disease_data/
|
|-- frontend/
|   |-- package.json
|   |-- tailwind.config.js
|   |-- index.html
|   `-- src/
|       |-- App.tsx
|       |-- components/
|       |   |-- ui/
|       |   |   |-- Button.tsx
|       |   |   |-- Card.tsx
|       |   |   |-- Input.tsx
|       |   |   |-- Avatar.tsx
|       |   |   `-- Spinner.tsx
|       |   |-- StatsCard.tsx
|       |   |-- RecentScansTable.tsx
|       |   |-- ProtectedRoute.tsx
|       |   `-- Layout.tsx
|       |-- pages/
|       |   |-- Signup.tsx
|       |   |-- Login.tsx
|       |   |-- Dashboard.tsx
|       |   |-- NewScan.tsx
|       |   |-- Analyzing.tsx
|       |   |-- Result.tsx
```

```
|       |   |-- History.tsx
|       |   |-- Profile.tsx
|       |   `-- About.tsx
|       |-- services/
|       |   `-- api.ts
|       |-- store/
|       |   `-- authStore.ts
|       |-- hooks/
|       `-- assets/
|
`-- docs/
    |-- SETUP.md
    |-- ARCHITECTURE.md
    |-- API_REFERENCE.md
    |-- DATABASE_SCHEMA.md
    |-- SECURITY.md
    |-- PERFORMANCE.md
    |-- TESTING_REPORT.md
    `-- DEMO_SCRIPT.md
```

Detailed Prompts

2.A Project Setup

PROMPT 56 — Initialize backend/ as a Python FastAPI project with requirements.txt (fastapi, uvicorn,...

Objective: Initialize `backend/` as a Python FastAPI project with `requirements.txt` (fastapi, uvicorn, sqlalchemy, psycopg2-binary, python-jose, passlib[bcrypt], python-multipart, pillow, tensorflow, pydantic-settings).

Why This Matters: Project-setup prompts establish the skeleton every later prompt in this module builds on. Getting folder names, config loading, and dependency lists right here prevents painful restructuring later.

Primary File(s): `backend/`, `requirements.txt`

Detailed Requirements

- Initialize `backend/` as a Python FastAPI project with `requirements.txt` (fastapi, uvicorn, sqlalchemy, psycopg2-binary, python-jose, passlib[bcrypt], python-multipart, pillow, tensorflow, pydantic-settings).
- Use the exact folder/file names given so later prompts (which reference these paths directly) do not break.
- Pin dependency versions in the relevant requirements/package file rather than leaving them unpinned.
- Add a short README or docstring at the top of the new file explaining its purpose.
- Keep secrets/config out of version control; use `.env` / `.env.example` as appropriate.

Technical Specification

No existing schema/API dependency — this is foundational scaffolding. Follow the folder tree in the project's master file-structure reference exactly.

Acceptance Criteria

- ☐ The new folder/file exists at the exact path specified.
- ☐ Running the relevant install command (`pip install -r requirements.txt` / `npm install`) succeeds with no errors.
- ☐ No secrets or credentials are committed to the repository.

Edge Cases & Notes

- If a dependency version conflicts with another already-pinned dependency, document the conflict and choose the lowest version that satisfies both rather than silently upgrading everything.

PROMPT 57 — Create the backend folder structure

Objective: Create the backend folder structure: `backend/app/{main.py, routes/, services/, models/, schemas/, core/, db/}` and `backend/app/disease_data/`.

Why This Matters: Project-setup prompts establish the skeleton every later prompt in this module builds on. Getting folder names, config loading, and dependency lists right here prevents painful restructuring later.

Primary File(s): `backend/app/{main.py, routes/, services/, models/, schemas/, core/, db/}`, `backend/app/disease_data/`

Detailed Requirements

- Create the backend folder structure: `backend/app/{main.py, routes/, services/, models/, schemas/, core/, db/}` and `backend/app/disease_data/`.

- Use the exact folder/file names given so later prompts (which reference these paths directly) do not break.
- Pin dependency versions in the relevant requirements/package file rather than leaving them unpinned.
- Add a short README or docstring at the top of the new file explaining its purpose.
- Keep secrets/config out of version control; use `.env` / `.env.example` as appropriate.

Technical Specification

No existing schema/API dependency — this is foundational scaffolding. Follow the folder tree in the project's master file-structure reference exactly.

Acceptance Criteria

- ☐ The new folder/file exists at the exact path specified.
- ☐ Running the relevant install command (`pip install -r requirements.txt` / `npm install`) succeeds with no errors.
- ☐ No secrets or credentials are committed to the repository.

Edge Cases & Notes

- If a dependency version conflicts with another already-pinned dependency, document the conflict and choose the lowest version that satisfies both rather than silently upgrading everything.

PROMPT 58 — Create backend/app/core/config.py using Pydantic Settings to load environment variables (D...

Objective: Create `backend/app/core/config.py` using Pydantic Settings to load environment variables (DATABASE_URL, JWT_SECRET, JWT_EXPIRY, MODEL_PATH).

Why This Matters: Project-setup prompts establish the skeleton every later prompt in this module builds on. Getting folder names, config loading, and dependency lists right here prevents painful restructuring later.

Primary File(s): `backend/app/core/config.py`

Detailed Requirements

- Create `backend/app/core/config.py` using Pydantic Settings to load environment variables (DATABASE_URL, JWT_SECRET, JWT_EXPIRY, MODEL_PATH).
- Use the exact folder/file names given so later prompts (which reference these paths directly) do not break.
- Pin dependency versions in the relevant requirements/package file rather than leaving them unpinned.
- Add a short README or docstring at the top of the new file explaining its purpose.
- Keep secrets/config out of version control; use `.env` / `.env.example` as appropriate.

Technical Specification

No existing schema/API dependency — this is foundational scaffolding. Follow the folder tree in the project's master file-structure reference exactly.

Acceptance Criteria

- ☐ The new folder/file exists at the exact path specified.
- ☐ Running the relevant install command (`pip install -r requirements.txt` / `npm install`) succeeds with no errors.
- ☐ No secrets or credentials are committed to the repository.

Edge Cases & Notes

- If a dependency version conflicts with another already-pinned dependency, document the conflict and choose the lowest version that satisfies both rather than silently upgrading everything.

PROMPT 59 — Set up backend/.env.example documenting required environment variables without real secrets

Objective: Set up `backend/.env.example` documenting required environment variables without real secrets.

Why This Matters: Project-setup prompts establish the skeleton every later prompt in this module builds on. Getting folder names, config loading, and dependency lists right here prevents painful restructuring later.

Primary File(s): `backend/.env.example`

Detailed Requirements

- Set up `backend/.env.example` documenting required environment variables without real secrets.
- Use the exact folder/file names given so later prompts (which reference these paths directly) do not break.
- Pin dependency versions in the relevant requirements/package file rather than leaving them unpinned.
- Add a short README or docstring at the top of the new file explaining its purpose.
- Keep secrets/config out of version control; use `.env` / `.env.example` as appropriate.

Technical Specification

No existing schema/API dependency — this is foundational scaffolding. Follow the folder tree in the project's master file-structure reference exactly.

Acceptance Criteria

- ☐ The new folder/file exists at the exact path specified.
- ☐ Running the relevant install command (`pip install -r requirements.txt` / `npm install`) succeeds with no errors.
- ☐ No secrets or credentials are committed to the repository.

Edge Cases & Notes

- If a dependency version conflicts with another already-pinned dependency, document the conflict and choose the lowest version that satisfies both rather than silently upgrading everything.

PROMPT 60 — Create backend/app/main.py with a basic FastAPI app instance, CORS middleware, and a healt...

Objective: Create `backend/app/main.py` with a basic FastAPI app instance, CORS middleware, and a health-check `GET /health` route.

Why This Matters: Project-setup prompts establish the skeleton every later prompt in this module builds on. Getting folder names, config loading, and dependency lists right here prevents painful restructuring later.

Primary File(s): `backend/app/main.py`, `GET /health`

Detailed Requirements

- Create `backend/app/main.py` with a basic FastAPI app instance, CORS middleware, and a health-check `GET /health` route.
- Use the exact folder/file names given so later prompts (which reference these paths directly) do not break.
- Pin dependency versions in the relevant requirements/package file rather than leaving them unpinned.

- Add a short README or docstring at the top of the new file explaining its purpose.
- Keep secrets/config out of version control; use `.env` / `.env.example` as appropriate.

Technical Specification

No existing schema/API dependency — this is foundational scaffolding. Follow the folder tree in the project's master file-structure reference exactly.

Acceptance Criteria

- ☐ The new folder/file exists at the exact path specified.
- ☐ Running the relevant install command (`pip install -r requirements.txt` / `npm install`) succeeds with no errors.
- ☐ No secrets or credentials are committed to the repository.

Edge Cases & Notes

- If a dependency version conflicts with another already-pinned dependency, document the conflict and choose the lowest version that satisfies both rather than silently upgrading everything.

2.B Database Schema & Migrations

PROMPT 61 — Set up the SQLAlchemy engine and session management in backend/app/db/session.py

Objective: Set up the SQLAlchemy engine and session management in `backend/app/db/session.py`.

Why This Matters: The database schema is the contract every other backend and frontend prompt relies on. A field added, renamed, or mistyped here causes cascading breakage across the auth, prediction, and history modules.

Primary File(s): `backend/app/db/session.py`

Detailed Requirements

- Set up the SQLAlchemy engine and session management in `backend/app/db/session.py`.
- Use SQLAlchemy's declarative `Base` and correct column types (`String`, `Float`, `Boolean`, `DateTime`, `ForeignKey`).
- Add `created_at` with a server-side default (`server_default=func.now()`), not an application-side default.
- Add explicit `ForeignKey` constraints and `relationship()` definitions with `back_populates` where two tables relate.
- Add a `__repr__` method to each model for easier debugging.
- Ensure the migration (Alembic) generated from this model is reviewed before being applied — autogenerate is a starting point, not the final migration.

Technical Specification

Refer to the full four-table schema (users, predictions, diseases, feedback) documented in docs/DATABASE_SCHEMA.md and the project report's Database Design chapter.

Acceptance Criteria

- ☐ Model class matches every field, type, and constraint listed in the technical specification above.
- ☐ `alembic upgrade head` applies the migration cleanly against a fresh database.
- ☐ A row can be inserted and queried back via the SQLAlchemy session in a quick manual test.

Edge Cases & Notes

- Decide and document the ON DELETE behaviour for foreign keys (e.g. CASCADE vs RESTRICT) rather than leaving it to the database default silently.
- Confirm `email` (on User) has a unique constraint and index, since login performance and duplicate-prevention both depend on it.

PROMPT 62 — Define the User model in backend/app/models/user.py with fields

Objective: Define the `User` model in `backend/app/models/user.py` with fields: id, name, email, password_hash, phone, location, profile_photo_url, preferred_crops, created_at, last_login_at.

Why This Matters: The database schema is the contract every other backend and frontend prompt relies on. A field added, renamed, or mistyped here causes cascading breakage across the auth, prediction, and history modules.

Primary File(s): `backend/app/models/user.py`

Detailed Requirements

- Define the `User` model in `backend/app/models/user.py` with fields: id, name, email, password_hash, phone, location, profile_photo_url, preferred_crops, created_at, last_login_at.
- Use SQLAlchemy's declarative `Base` and correct column types (`String`, `Float`, `Boolean`, `DateTime`, `ForeignKey`).
- Add `created_at` with a server-side default (`server_default=func.now()`), not an application-side default.
- Add explicit `ForeignKey` constraints and `relationship()` definitions with `back_populates` where two tables relate.
- Add a `__repr__` method to each model for easier debugging.
- Ensure the migration (Alembic) generated from this model is reviewed before being applied — autogenerate is a starting point, not the final migration.

Technical Specification

User table columns — id (PK, UUID/int) · name (str, required) · email (str, unique, indexed, required) · password_hash (str, required, bcrypt) · phone (str, nullable) · location (str, nullable) · profile_photo_url (str, nullable) · preferred_crops (str/array, nullable) · created_at (datetime, server default now) · last_login_at (datetime, nullable)

Acceptance Criteria

- ☐ Model class matches every field, type, and constraint listed in the technical specification above.
- ☐ `alembic upgrade head` applies the migration cleanly against a fresh database.
- ☐ A row can be inserted and queried back via the SQLAlchemy session in a quick manual test.

Edge Cases & Notes

- Decide and document the ON DELETE behaviour for foreign keys (e.g. CASCADE vs RESTRICT) rather than leaving it to the database default silently.
- Confirm `email` (on User) has a unique constraint and index, since login performance and duplicate-prevention both depend on it.

PROMPT 63 — Define the Prediction model in backend/app/models/prediction.py with fields

Objective: Define the `Prediction` model in `backend/app/models/prediction.py` with fields: id, user_id (FK), image_url, crop, disease, confidence, is_confident, created_at.

Why This Matters: The database schema is the contract every other backend and frontend prompt relies on. A field added, renamed, or mistyped here causes cascading breakage across the auth, prediction, and history modules.

Primary File(s): `backend/app/models/prediction.py`

Detailed Requirements

- Define the `Prediction` model in `backend/app/models/prediction.py` with fields: id, user_id (FK), image_url, crop, disease, confidence, is_confident, created_at.
- Use SQLAlchemy's declarative `Base` and correct column types (`String`, `Float`, `Boolean`, `DateTime`, `ForeignKey`).
- Add `created_at` with a server-side default (`server_default=func.now()`), not an application-side default.
- Add explicit `ForeignKey` constraints and `relationship()` definitions with `back_populates` where two tables relate.
- Add a `__repr__` method to each model for easier debugging.

- Ensure the migration (Alembic) generated from this model is reviewed before being applied — autogenerate is a starting point, not the final migration.

Technical Specification

Prediction table columns — id (PK) · user_id (FK -> users.id, indexed, required) · image_url (str, nullable) · crop (str, required) · disease (str, required) · confidence (float 0-1, required) · is_confident (bool, required) · created_at (datetime, server default now)

Acceptance Criteria

- ☐ Model class matches every field, type, and constraint listed in the technical specification above.
- ☐ `alembic upgrade head` applies the migration cleanly against a fresh database.
- ☐ A row can be inserted and queried back via the SQLAlchemy session in a quick manual test.

Edge Cases & Notes

- Decide and document the ON DELETE behaviour for foreign keys (e.g. CASCADE vs RESTRICT) rather than leaving it to the database default silently.
- Confirm `email` (on User) has a unique constraint and index, since login performance and duplicate-prevention both depend on it.

PROMPT 64 — Define the Disease model in backend/app/models/disease.py with fields

Objective: Define the `Disease` model in `backend/app/models/disease.py` with fields: id, crop, disease_name, description, symptoms, prevention, management.

Why This Matters: The database schema is the contract every other backend and frontend prompt relies on. A field added, renamed, or mistyped here causes cascading breakage across the auth, prediction, and history modules.

Primary File(s): `backend/app/models/disease.py`

Detailed Requirements

- Define the `Disease` model in `backend/app/models/disease.py` with fields: id, crop, disease_name, description, symptoms, prevention, management.
- Use SQLAlchemy's declarative `Base` and correct column types (`String`, `Float`, `Boolean`, `DateTime`, `ForeignKey`).
- Add `created_at` with a server-side default (`server_default=func.now()`), not an application-side default.
- Add explicit `ForeignKey` constraints and `relationship()` definitions with `back_populates` where two tables relate.
- Add a `__repr__` method to each model for easier debugging.
- Ensure the migration (Alembic) generated from this model is reviewed before being applied — autogenerate is a starting point, not the final migration.

Technical Specification

Disease table columns — id (PK) · crop (str, required) · disease_name (str, required) · description (text) · symptoms (text/array) · prevention (text/array) · management (text/array)

Acceptance Criteria

- ☐ Model class matches every field, type, and constraint listed in the technical specification above.
- ☐ `alembic upgrade head` applies the migration cleanly against a fresh database.
- ☐ A row can be inserted and queried back via the SQLAlchemy session in a quick manual test.

Edge Cases & Notes

- Decide and document the ON DELETE behaviour for foreign keys (e.g. CASCADE vs RESTRICT) rather than leaving it to the database default silently.
- Confirm `email` (on User) has a unique constraint and index, since login performance and duplicate-prevention both depend on it.

PROMPT 65 — Define the Feedback model in backend/app/models/feedback.py with fields

Objective: Define the `Feedback` model in `backend/app/models/feedback.py` with fields: id, prediction_id (FK), user_id (FK), actual_disease, is_correct, comment, created_at.

Why This Matters: The database schema is the contract every other backend and frontend prompt relies on. A field added, renamed, or mistyped here causes cascading breakage across the auth, prediction, and history modules.

Primary File(s): `backend/app/models/feedback.py`

Detailed Requirements

- Define the `Feedback` model in `backend/app/models/feedback.py` with fields: id, prediction_id (FK), user_id (FK), actual_disease, is_correct, comment, created_at.
- Use SQLAlchemy's declarative `Base` and correct column types (`String`, `Float`, `Boolean`, `DateTime`, `ForeignKey`).
- Add `created_at` with a server-side default (`server_default=func.now()`), not an application-side default.
- Add explicit `ForeignKey` constraints and `relationship()` definitions with `back_populates` where two tables relate.
- Add a `__repr__` method to each model for easier debugging.
- Ensure the migration (Alembic) generated from this model is reviewed before being applied — autogenerate is a starting point, not the final migration.

Technical Specification

Feedback table columns — id (PK) · prediction_id (FK -> predictions.id, required) · user_id (FK -> users.id, required) · actual_disease (str, nullable) · is_correct (bool, required) · comment (text, nullable) · created_at (datetime, server default now)

Acceptance Criteria

- ☐ Model class matches every field, type, and constraint listed in the technical specification above.
- ☐ `alembic upgrade head` applies the migration cleanly against a fresh database.
- ☐ A row can be inserted and queried back via the SQLAlchemy session in a quick manual test.

Edge Cases & Notes

- Decide and document the ON DELETE behaviour for foreign keys (e.g. CASCADE vs RESTRICT) rather than leaving it to the database default silently.
- Confirm `email` (on User) has a unique constraint and index, since login performance and duplicate-prevention both depend on it.

PROMPT 66 — Set up Alembic for migrations in backend/alembic/ and generate the initial migration cover...

Objective: Set up Alembic for migrations in `backend/alembic/` and generate the initial migration covering all four tables.

Why This Matters: The database schema is the contract every other backend and frontend prompt relies on. A field added, renamed, or mistyped here causes cascading breakage across the auth, prediction, and history modules.

Primary File(s): `backend/alembic/`

Detailed Requirements

- Set up Alembic for migrations in `backend/alembic/` and generate the initial migration covering all four tables.
- Use SQLAlchemy's declarative `Base` and correct column types (`String`, `Float`, `Boolean`, `DateTime`, `ForeignKey`).
- Add `created_at` with a server-side default (`server_default=func.now()`), not an application-side default.
- Add explicit `ForeignKey` constraints and `relationship()` definitions with `back_populates` where two tables relate.
- Add a `__repr__` method to each model for easier debugging.
- Ensure the migration (Alembic) generated from this model is reviewed before being applied — autogenerate is a starting point, not the final migration.

Technical Specification

Refer to the full four-table schema (users, predictions, diseases, feedback) documented in docs/DATABASE_SCHEMA.md and the project report's Database Design chapter.

Acceptance Criteria

- ☐ Model class matches every field, type, and constraint listed in the technical specification above.
- ☐ `alembic upgrade head` applies the migration cleanly against a fresh database.
- ☐ A row can be inserted and queried back via the SQLAlchemy session in a quick manual test.

Edge Cases & Notes

- Decide and document the ON DELETE behaviour for foreign keys (e.g. CASCADE vs RESTRICT) rather than leaving it to the database default silently.
- Confirm `email` (on User) has a unique constraint and index, since login performance and duplicate-prevention both depend on it.

PROMPT 67 — Write backend/app/db/seed_diseases.py to populate the diseases table with structured dat...

Objective: Write `backend/app/db/seed_diseases.py` to populate the `diseases` table with structured data for all 8 classes (description, symptoms, prevention, management).

Why This Matters: The database schema is the contract every other backend and frontend prompt relies on. A field added, renamed, or mistyped here causes cascading breakage across the auth, prediction, and history modules.

Primary File(s): `backend/app/db/seed_diseases.py`

Detailed Requirements

- Write `backend/app/db/seed_diseases.py` to populate the `diseases` table with structured data for all 8 classes (description, symptoms, prevention, management).
- Use SQLAlchemy's declarative `Base` and correct column types (`String`, `Float`, `Boolean`, `DateTime`, `ForeignKey`).

- Add `created_at` with a server-side default (`server_default=func.now()`), not an application-side default.
- Add explicit `ForeignKey` constraints and `relationship()` definitions with `back_populates` where two tables relate.
- Add a `__repr__` method to each model for easier debugging.
- Ensure the migration (Alembic) generated from this model is reviewed before being applied — autogenerate is a starting point, not the final migration.

Technical Specification

Disease table columns — id (PK) · crop (str, required) · disease_name (str, required) · description (text) · symptoms (text/array) · prevention (text/array) · management (text/array)

Acceptance Criteria

- ☐ Model class matches every field, type, and constraint listed in the technical specification above.
- ☐ `alembic upgrade head` applies the migration cleanly against a fresh database.
- ☐ A row can be inserted and queried back via the SQLAlchemy session in a quick manual test.

Edge Cases & Notes

- Decide and document the ON DELETE behaviour for foreign keys (e.g. CASCADE vs RESTRICT) rather than leaving it to the database default silently.
- Confirm `email` (on User) has a unique constraint and index, since login performance and duplicate-prevention both depend on it.

PROMPT 68 — Write backend/scripts/reset_db.py to drop and recreate all tables and re-seed disease data…

Objective: Write `backend/scripts/reset_db.py` to drop and recreate all tables and re-seed disease data, for local development.

Why This Matters: The database schema is the contract every other backend and frontend prompt relies on. A field added, renamed, or mistyped here causes cascading breakage across the auth, prediction, and history modules.

Primary File(s): `backend/scripts/reset_db.py`

Detailed Requirements

- Write `backend/scripts/reset_db.py` to drop and recreate all tables and re-seed disease data, for local development.
- Use SQLAlchemy's declarative `Base` and correct column types (`String`, `Float`, `Boolean`, `DateTime`, `ForeignKey`).
- Add `created_at` with a server-side default (`server_default=func.now()`), not an application-side default.
- Add explicit `ForeignKey` constraints and `relationship()` definitions with `back_populates` where two tables relate.
- Add a `__repr__` method to each model for easier debugging.
- Ensure the migration (Alembic) generated from this model is reviewed before being applied — autogenerate is a starting point, not the final migration.

Technical Specification

Disease table columns — id (PK) · crop (str, required) · disease_name (str, required) · description (text) · symptoms (text/array) · prevention (text/array) · management (text/array)

Acceptance Criteria

- ☐ Model class matches every field, type, and constraint listed in the technical specification above.
- ☐ `alembic upgrade head` applies the migration cleanly against a fresh database.
- ☐ A row can be inserted and queried back via the SQLAlchemy session in a quick manual test.

Edge Cases & Notes

- Decide and document the ON DELETE behaviour for foreign keys (e.g. CASCADE vs RESTRICT) rather than leaving it to the database default silently.
- Confirm `email` (on User) has a unique constraint and index, since login performance and duplicate-prevention both depend on it.

2.C Authentication Module

PROMPT 69 — Create backend/app/core/security.py with password hashing/verification functions using pas...

Objective: Create `backend/app/core/security.py` with password hashing/verification functions using passlib's bcrypt context.

Why This Matters: Authentication is the security boundary for the entire personalized-account system described in the project report. Every other protected endpoint trusts that this module correctly verifies identity — a flaw here compromises every user's private data.

Primary File(s): `backend/app/core/security.py`

Detailed Requirements

- Create `backend/app/core/security.py` with password hashing/verification functions using passlib's bcrypt context.
- Never store or log a raw password — only the bcrypt hash, via passlib's `CryptContext`.
- JWT payload should contain the user's ID and an expiry (`exp`) claim; sign with a secret loaded from environment config, never hard-coded.
- Return a generic “invalid credentials” error for both ‘user not found’ and ‘wrong password’ cases, so the API never reveals which emails are registered.
- Validate email format and a minimum password length/strength on the `UserCreate` schema before hashing.
- Write both a success-path and a failure-path test for this endpoint before considering it complete.

Technical Specification

Request/response governed by the `UserCreate` / `UserLogin` / `Token` Pydantic schemas. Response on success: `{ "access_token": "<jwt>", "token_type": "bearer" }`. Status codes: 201 (register success), 200 (login success), 400 (validation error), 401 (bad credentials), 409 (duplicate email on register).

Acceptance Criteria

- ☐ A new user can register, and the returned JWT successfully authenticates a subsequent protected request.
- ☐ Registering with an already-used email returns a 409, not a 500 or a silent duplicate.
- ☐ Logging in with a wrong password returns 401 with no indication of which field was wrong.
- ☐ `backend/tests/test_auth.py` passes for this endpoint.

Edge Cases & Notes

- Reject empty-string or whitespace-only passwords explicitly rather than hashing them.
- Confirm token expiry is actually enforced (an expired token is rejected, not silently accepted).

PROMPT 70 — Add JWT encode/decode helper functions in backend/app/core/security.py using python-jose, ...

Objective: Add JWT encode/decode helper functions in `backend/app/core/security.py` using python-jose, including expiry handling.

Why This Matters: Authentication is the security boundary for the entire personalized-account system described in the project report. Every other protected endpoint trusts that this module correctly verifies identity — a flaw here compromises every user's private data.

Primary File(s): `backend/app/core/security.py`

Detailed Requirements

- Add JWT encode/decode helper functions in `backend/app/core/security.py` using python-jose, including expiry handling.
- Never store or log a raw password — only the bcrypt hash, via passlib's `CryptContext`.
- JWT payload should contain the user's ID and an expiry (`exp`) claim; sign with a secret loaded from environment config, never hard-coded.
- Return a generic “invalid credentials” error for both ‘user not found’ and ‘wrong password’ cases, so the API never reveals which emails are registered.
- Validate email format and a minimum password length/strength on the `UserCreate` schema before hashing.
- Write both a success-path and a failure-path test for this endpoint before considering it complete.

Technical Specification

Request/response governed by the `UserCreate` / `UserLogin` / `Token` Pydantic schemas. Response on success: `{ "access_token": "<jwt>", "token_type": "bearer" }`. Status codes: 201 (register success), 200 (login success), 400 (validation error), 401 (bad credentials), 409 (duplicate email on register).

Acceptance Criteria

- ☐ A new user can register, and the returned JWT successfully authenticates a subsequent protected request.
- ☐ Registering with an already-used email returns a 409, not a 500 or a silent duplicate.
- ☐ Logging in with a wrong password returns 401 with no indication of which field was wrong.
- ☐ `backend/tests/test_auth.py` passes for this endpoint.

Edge Cases & Notes

- Reject empty-string or whitespace-only passwords explicitly rather than hashing them.
- Confirm token expiry is actually enforced (an expired token is rejected, not silently accepted).

PROMPT 71 — Define Pydantic schemas UserCreate, UserLogin, UserOut, Token in backend/app/schema...

Objective: Define Pydantic schemas `UserCreate`, `UserLogin`, `UserOut`, `Token` in `backend/app/schemas/auth.py`.

Why This Matters: Authentication is the security boundary for the entire personalized-account system described in the project report. Every other protected endpoint trusts that this module correctly verifies identity — a flaw here compromises every user's private data.

Primary File(s): `backend/app/schemas/auth.py`

Detailed Requirements

- Define Pydantic schemas `UserCreate`, `UserLogin`, `UserOut`, `Token` in `backend/app/schemas/auth.py`.
- Never store or log a raw password — only the bcrypt hash, via passlib's `CryptContext`.
- JWT payload should contain the user's ID and an expiry (`exp`) claim; sign with a secret loaded from environment config, never hard-coded.
- Return a generic “invalid credentials” error for both ‘user not found’ and ‘wrong password’ cases, so the API never reveals which emails are registered.

- Validate email format and a minimum password length/strength on the `UserCreate` schema before hashing.
- Write both a success-path and a failure-path test for this endpoint before considering it complete.

Technical Specification

Request/response governed by the `UserCreate` / `UserLogin` / `Token` Pydantic schemas. Response on success: `{ "access_token": "<jwt>", "token_type": "bearer" }`. Status codes: 201 (register success), 200 (login success), 400 (validation error), 401 (bad credentials), 409 (duplicate email on register).

Acceptance Criteria

- ☐ A new user can register, and the returned JWT successfully authenticates a subsequent protected request.
- ☐ Registering with an already-used email returns a 409, not a 500 or a silent duplicate.
- ☐ Logging in with a wrong password returns 401 with no indication of which field was wrong.
- ☐ `backend/tests/test_auth.py` passes for this endpoint.

Edge Cases & Notes

- Reject empty-string or whitespace-only passwords explicitly rather than hashing them.
- Confirm token expiry is actually enforced (an expired token is rejected, not silently accepted).

PROMPT 72 — Implement POST /auth/register in backend/app/routes/auth.py

Objective: Implement `POST /auth/register` in `backend/app/routes/auth.py`: validate input, hash password, create user row, issue JWT.

Why This Matters: Authentication is the security boundary for the entire personalized-account system described in the project report. Every other protected endpoint trusts that this module correctly verifies identity — a flaw here compromises every user's private data.

Primary File(s): `POST /auth/register`, `backend/app/routes/auth.py`

Detailed Requirements

- Implement `POST /auth/register` in `backend/app/routes/auth.py`: validate input, hash password, create user row, issue JWT.
- Never store or log a raw password — only the bcrypt hash, via passlib's `CryptContext`.
- JWT payload should contain the user's ID and an expiry (`exp`) claim; sign with a secret loaded from environment config, never hard-coded.
- Return a generic “invalid credentials” error for both ‘user not found’ and ‘wrong password’ cases, so the API never reveals which emails are registered.
- Validate email format and a minimum password length/strength on the `UserCreate` schema before hashing.
- Write both a success-path and a failure-path test for this endpoint before considering it complete.

Technical Specification

Route: POST /auth/register. Request/response governed by the `UserCreate` / `UserLogin` / `Token` Pydantic schemas. Response on success: `{ "access_token": "<jwt>", "token_type": "bearer" }`. Status codes: 201 (register success), 200 (login success), 400 (validation error), 401 (bad credentials), 409 (duplicate email on register).

Acceptance Criteria

- ☐ A new user can register, and the returned JWT successfully authenticates a subsequent protected request.
- ☐ Registering with an already-used email returns a 409, not a 500 or a silent duplicate.

- ☐ Logging in with a wrong password returns 401 with no indication of which field was wrong.
- ☐ `backend/tests/test_auth.py` passes for this endpoint.

Edge Cases & Notes

- Reject empty-string or whitespace-only passwords explicitly rather than hashing them.
- Confirm token expiry is actually enforced (an expired token is rejected, not silently accepted).

PROMPT 73 — Implement POST /auth/login in backend/app/routes/auth.py

Objective: Implement `POST /auth/login` in `backend/app/routes/auth.py`: verify credentials, update `last_login_at`, issue JWT.

Why This Matters: Authentication is the security boundary for the entire personalized-account system described in the project report. Every other protected endpoint trusts that this module correctly verifies identity — a flaw here compromises every user's private data.

Primary File(s): `POST /auth/login`, `backend/app/routes/auth.py`

Detailed Requirements

- Implement `POST /auth/login` in `backend/app/routes/auth.py`: verify credentials, update `last_login_at`, issue JWT.
- Never store or log a raw password — only the bcrypt hash, via passlib's `CryptContext`.
- JWT payload should contain the user's ID and an expiry (`exp`) claim; sign with a secret loaded from environment config, never hard-coded.
- Return a generic “invalid credentials” error for both ‘user not found’ and ‘wrong password’ cases, so the API never reveals which emails are registered.
- Validate email format and a minimum password length/strength on the `UserCreate` schema before hashing.
- Write both a success-path and a failure-path test for this endpoint before considering it complete.

Technical Specification

Route: POST /auth/login. Request/response governed by the `UserCreate` / `UserLogin` / `Token` Pydantic schemas. Response on success: `{ "access_token": "<jwt>", "token_type": "bearer" }`. Status codes: 201 (register success), 200 (login success), 400 (validation error), 401 (bad credentials), 409 (duplicate email on register).

Acceptance Criteria

- ☐ A new user can register, and the returned JWT successfully authenticates a subsequent protected request.
- ☐ Registering with an already-used email returns a 409, not a 500 or a silent duplicate.
- ☐ Logging in with a wrong password returns 401 with no indication of which field was wrong.
- ☐ `backend/tests/test_auth.py` passes for this endpoint.

Edge Cases & Notes

- Reject empty-string or whitespace-only passwords explicitly rather than hashing them.
- Confirm token expiry is actually enforced (an expired token is rejected, not silently accepted).

PROMPT 74 — Create a get_current_user dependency in backend/app/core/dependencies.py that decodes th...

Objective: Create a `get_current_user` dependency in `backend/app/core/dependencies.py` that decodes the JWT from the Authorization header and loads the user from the database.

Why This Matters: Authentication is the security boundary for the entire personalized-account system described in the project report. Every other protected endpoint trusts that this module correctly verifies identity — a flaw here compromises every user's private data.

Primary File(s): `backend/app/core/dependencies.py`

Detailed Requirements

- Create a `get_current_user` dependency in `backend/app/core/dependencies.py` that decodes the JWT from the Authorization header and loads the user from the database.
- Never store or log a raw password — only the bcrypt hash, via passlib's `CryptContext`.
- JWT payload should contain the user's ID and an expiry (`exp`) claim; sign with a secret loaded from environment config, never hard-coded.
- Return a generic “invalid credentials” error for both ‘user not found’ and ‘wrong password’ cases, so the API never reveals which emails are registered.
- Validate email format and a minimum password length/strength on the `UserCreate` schema before hashing.
- Write both a success-path and a failure-path test for this endpoint before considering it complete.

Technical Specification

Request/response governed by the `UserCreate` / `UserLogin` / `Token` Pydantic schemas. Response on success: `{ "access_token": "<jwt>", "token_type": "bearer" }`. Status codes: 201 (register success), 200 (login success), 400 (validation error), 401 (bad credentials), 409 (duplicate email on register).

Acceptance Criteria

- ☐ A new user can register, and the returned JWT successfully authenticates a subsequent protected request.
- ☐ Registering with an already-used email returns a 409, not a 500 or a silent duplicate.
- ☐ Logging in with a wrong password returns 401 with no indication of which field was wrong.
- ☐ `backend/tests/test_auth.py` passes for this endpoint.

Edge Cases & Notes

- Reject empty-string or whitespace-only passwords explicitly rather than hashing them.
- Confirm token expiry is actually enforced (an expired token is rejected, not silently accepted).

PROMPT 75 — Add proper 401 error handling for missing/invalid/expired tokens across all protected routes

Objective: Add proper 401 error handling for missing/invalid/expired tokens across all protected routes.

Why This Matters: Authentication is the security boundary for the entire personalized-account system described in the project report. Every other protected endpoint trusts that this module correctly verifies identity — a flaw here compromises every user's private data.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add proper 401 error handling for missing/invalid/expired tokens across all protected routes.
- Never store or log a raw password — only the bcrypt hash, via passlib's `CryptContext`.
- JWT payload should contain the user's ID and an expiry (`exp`) claim; sign with a secret loaded from environment config, never hard-coded.
- Return a generic “invalid credentials” error for both ‘user not found’ and ‘wrong password’ cases, so the API never reveals which emails are registered.

- Validate email format and a minimum password length/strength on the `UserCreate` schema before hashing.
- Write both a success-path and a failure-path test for this endpoint before considering it complete.

Technical Specification

Request/response governed by the `UserCreate` / `UserLogin` / `Token` Pydantic schemas. Response on success: `{ "access_token": "<jwt>", "token_type": "bearer" }`. Status codes: 201 (register success), 200 (login success), 400 (validation error), 401 (bad credentials), 409 (duplicate email on register).

Acceptance Criteria

- ☐ A new user can register, and the returned JWT successfully authenticates a subsequent protected request.
- ☐ Registering with an already-used email returns a 409, not a 500 or a silent duplicate.
- ☐ Logging in with a wrong password returns 401 with no indication of which field was wrong.
- ☐ `backend/tests/test_auth.py` passes for this endpoint.

Edge Cases & Notes

- Reject empty-string or whitespace-only passwords explicitly rather than hashing them.
- Confirm token expiry is actually enforced (an expired token is rejected, not silently accepted).

PROMPT 76 — Write backend/tests/test_auth.py covering register success, duplicate-email rejection, log...

Objective: Write `backend/tests/test_auth.py` covering register success, duplicate-email rejection, login success, and login failure with wrong password.

Why This Matters: Authentication is the security boundary for the entire personalized-account system described in the project report. Every other protected endpoint trusts that this module correctly verifies identity — a flaw here compromises every user's private data.

Primary File(s): `backend/tests/test_auth.py`

Detailed Requirements

- Write `backend/tests/test_auth.py` covering register success, duplicate-email rejection, login success, and login failure with wrong password.
- Never store or log a raw password — only the bcrypt hash, via passlib's `CryptContext`.
- JWT payload should contain the user's ID and an expiry (`exp`) claim; sign with a secret loaded from environment config, never hard-coded.
- Return a generic “invalid credentials” error for both ‘user not found’ and ‘wrong password’ cases, so the API never reveals which emails are registered.
- Validate email format and a minimum password length/strength on the `UserCreate` schema before hashing.
- Write both a success-path and a failure-path test for this endpoint before considering it complete.

Technical Specification

Request/response governed by the `UserCreate` / `UserLogin` / `Token` Pydantic schemas. Response on success: `{ "access_token": "<jwt>", "token_type": "bearer" }`. Status codes: 201 (register success), 200 (login success), 400 (validation error), 401 (bad credentials), 409 (duplicate email on register).

Acceptance Criteria

- ☐ A new user can register, and the returned JWT successfully authenticates a subsequent protected request.

- ☐ Registering with an already-used email returns a 409, not a 500 or a silent duplicate.
- ☐ Logging in with a wrong password returns 401 with no indication of which field was wrong.
- ☐ `backend/tests/test_auth.py` passes for this endpoint.

Edge Cases & Notes

- Reject empty-string or whitespace-only passwords explicitly rather than hashing them.
- Confirm token expiry is actually enforced (an expired token is rejected, not silently accepted).

PROMPT 77 — Add email-format and password-strength validation to the UserCreate schema

Objective: Add email-format and password-strength validation to the `UserCreate` schema.

Why This Matters: Authentication is the security boundary for the entire personalized-account system described in the project report. Every other protected endpoint trusts that this module correctly verifies identity — a flaw here compromises every user's private data.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add email-format and password-strength validation to the `UserCreate` schema.
- Never store or log a raw password — only the bcrypt hash, via passlib's `CryptContext`.
- JWT payload should contain the user's ID and an expiry (`exp`) claim; sign with a secret loaded from environment config, never hard-coded.
- Return a generic “invalid credentials” error for both ‘user not found’ and ‘wrong password’ cases, so the API never reveals which emails are registered.
- Validate email format and a minimum password length/strength on the `UserCreate` schema before hashing.
- Write both a success-path and a failure-path test for this endpoint before considering it complete.

Technical Specification

Request/response governed by the `UserCreate` / `UserLogin` / `Token` Pydantic schemas. Response on success: `{ "access_token": "<jwt>", "token_type": "bearer" }`. Status codes: 201 (register success), 200 (login success), 400 (validation error), 401 (bad credentials), 409 (duplicate email on register).

Acceptance Criteria

- ☐ A new user can register, and the returned JWT successfully authenticates a subsequent protected request.
- ☐ Registering with an already-used email returns a 409, not a 500 or a silent duplicate.
- ☐ Logging in with a wrong password returns 401 with no indication of which field was wrong.
- ☐ `backend/tests/test_auth.py` passes for this endpoint.

Edge Cases & Notes

- Reject empty-string or whitespace-only passwords explicitly rather than hashing them.
- Confirm token expiry is actually enforced (an expired token is rejected, not silently accepted).

PROMPT 78 — Add rate-limiting or basic brute-force protection notes/middleware stub on the /auth/login...

Objective: Add rate-limiting or basic brute-force protection notes/middleware stub on the `/auth/login` route.

Why This Matters: Authentication is the security boundary for the entire personalized-account system described in the project report. Every other protected endpoint trusts that this module correctly verifies identity — a flaw here compromises every user's private data.

Primary File(s): `/auth/login`

Detailed Requirements

- Add rate-limiting or basic brute-force protection notes/middleware stub on the `/auth/login` route.
- Never store or log a raw password — only the bcrypt hash, via passlib's `CryptContext`.
- JWT payload should contain the user's ID and an expiry (`exp`) claim; sign with a secret loaded from environment config, never hard-coded.
- Return a generic “invalid credentials” error for both ‘user not found’ and ‘wrong password’ cases, so the API never reveals which emails are registered.
- Validate email format and a minimum password length/strength on the `UserCreate` schema before hashing.
- Write both a success-path and a failure-path test for this endpoint before considering it complete.

Technical Specification

Request/response governed by the `UserCreate` / `UserLogin` / `Token` Pydantic schemas. Response on success: `{ "access_token": "<jwt>", "token_type": "bearer" }`. Status codes: 201 (register success), 200 (login success), 400 (validation error), 401 (bad credentials), 409 (duplicate email on register).

Acceptance Criteria

- ☐ A new user can register, and the returned JWT successfully authenticates a subsequent protected request.
- ☐ Registering with an already-used email returns a 409, not a 500 or a silent duplicate.
- ☐ Logging in with a wrong password returns 401 with no indication of which field was wrong.
- ☐ `backend/tests/test_auth.py` passes for this endpoint.

Edge Cases & Notes

- Reject empty-string or whitespace-only passwords explicitly rather than hashing them.
- Confirm token expiry is actually enforced (an expired token is rejected, not silently accepted).

2.D User Profile Module

PROMPT 79 — Define Pydantic schemas UserProfileOut and UserProfileUpdate in backend/app/schemas/use...

Objective: Define Pydantic schemas `UserProfileOut` and `UserProfileUpdate` in `backend/app/schemas/user.py`.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `backend/app/schemas/user.py`

Detailed Requirements

- Define Pydantic schemas `UserProfileOut` and `UserProfileUpdate` in `backend/app/schemas/user.py`.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

User table columns — id (PK, UUID/int) · name (str, required) · email (str, unique, indexed, required) · password_hash (str, required, bcrypt) · phone (str, nullable) · location (str, nullable) · profile_photo_url (str, nullable) · preferred_crops (str/array, nullable) · created_at (datetime, server default now) · last_login_at (datetime, nullable)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 80 — Implement GET /users/me in backend/app/routes/users.py returning the current authenticat...

Objective: Implement `GET /users/me` in `backend/app/routes/users.py` returning the current authenticated user's profile.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `GET /users/me`, `backend/app/routes/users.py`

Detailed Requirements

- Implement `GET /users/me` in `backend/app/routes/users.py` returning the current authenticated user's profile.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Route: GET /users/me. User table columns — id (PK, UUID/int) · name (str, required) · email (str, unique, indexed, required) · password_hash (str, required, bcrypt) · phone (str, nullable) · location (str, nullable) · profile_photo_url (str, nullable) · preferred_crops (str/array, nullable) · created_at (datetime, server default now) · last_login_at (datetime, nullable)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 81 — Implement PUT /users/me allowing the user to update name, phone, location, and preferred_c...

Objective: Implement `PUT /users/me` allowing the user to update name, phone, location, and preferred_crops.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `PUT /users/me`

Detailed Requirements

- Implement `PUT /users/me` allowing the user to update name, phone, location, and preferred_crops.

- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Route: PUT /users/me. User table columns — id (PK, UUID/int) · name (str, required) · email (str, unique, indexed, required) · password_hash (str, required, bcrypt) · phone (str, nullable) · location (str, nullable) · profile_photo_url (str, nullable) · preferred_crops (str/array, nullable) · created_at (datetime, server default now) · last_login_at (datetime, nullable)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 82 — Implement POST /users/me/photo accepting a profile-photo upload, storing it, and updating ...

Objective: Implement `POST /users/me/photo` accepting a profile-photo upload, storing it, and updating `profile_photo_url`.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `POST /users/me/photo`

Detailed Requirements

- Implement `POST /users/me/photo` accepting a profile-photo upload, storing it, and updating `profile_photo_url`.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Route: POST /users/me/photo. User table columns — id (PK, UUID/int) · name (str, required) · email (str, unique, indexed, required) · password_hash (str, required, bcrypt) · phone (str, nullable) · location (str, nullable) · profile_photo_url (str, nullable) · preferred_crops (str/array, nullable) · created_at (datetime, server default now) · last_login_at (datetime, nullable)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 83 — Add default-avatar generation logic (initials-based) for users without an uploaded photo

Objective: Add default-avatar generation logic (initials-based) for users without an uploaded photo.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add default-avatar generation logic (initials-based) for users without an uploaded photo.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

User table columns — id (PK, UUID/int) · name (str, required) · email (str, unique, indexed, required) · password_hash (str, required, bcrypt) · phone (str, nullable) · location (str, nullable) · profile_photo_url (str, nullable) · preferred_crops (str/array, nullable) · created_at (datetime, server default now) · last_login_at (datetime, nullable)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.

- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 84 — Implement GET /users/me/stats returning total scans, diseases-detected count, and healthy-...

Objective: Implement `GET /users/me/stats` returning total scans, diseases-detected count, and healthy-count for the dashboard cards.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `GET /users/me/stats`

Detailed Requirements

- Implement `GET /users/me/stats` returning total scans, diseases-detected count, and healthy-count for the dashboard cards.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Route: GET /users/me/stats. Disease table columns — id (PK) · crop (str, required) · disease_name (str, required) · description (text) · symptoms (text/array) · prevention (text/array) · management (text/array)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 85 — Write backend/tests/test_users.py covering profile fetch, profile update, and stats calcul...

Objective: Write `backend/tests/test_users.py` covering profile fetch, profile update, and stats calculation.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `backend/tests/test_users.py`

Detailed Requirements

- Write `backend/tests/test_users.py` covering profile fetch, profile update, and stats calculation.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Response/request shape should match the corresponding Pydantic schema already defined for this module.

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

2.E Prediction / Inference Module

PROMPT 86 — Create backend/app/services/model_service.py that loads crop_disease_model.keras and cl...

Objective: Create `backend/app/services/model_service.py` that loads `crop_disease_model.keras` and `class_names.json` once at application startup (singleton pattern).

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `backend/app/services/model_service.py`, `crop_disease_model.keras`, `class_names.json`

Detailed Requirements

- Create `backend/app/services/model_service.py` that loads `crop_disease_model.keras` and `class_names.json` once at application startup (singleton pattern).
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Response/request shape should match the corresponding Pydantic schema already defined for this module.

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 87 — Create backend/app/services/preprocessing_service.py replicating the exact resize/normaliz...

Objective: Create `backend/app/services/preprocessing_service.py` replicating the exact resize/normalize logic used during training, applied to incoming uploaded images.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `backend/app/services/preprocessing_service.py`

Detailed Requirements

- Create `backend/app/services/preprocessing_service.py` replicating the exact resize/normalize logic used during training, applied to incoming uploaded images.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Response/request shape should match the corresponding Pydantic schema already defined for this module.

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 88 — Define Pydantic schema PredictionOut in backend/app/schemas/prediction.py matching the d...

Objective: Define Pydantic schema `PredictionOut` in `backend/app/schemas/prediction.py` matching the documented API response shape.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `backend/app/schemas/prediction.py`

Detailed Requirements

- Define Pydantic schema `PredictionOut` in `backend/app/schemas/prediction.py` matching the documented API response shape.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Prediction table columns — id (PK) · user_id (FK -> users.id, indexed, required) · image_url (str, nullable) · crop (str, required) · disease (str, required) · confidence (float 0-1, required) · is_confident (bool, required) · created_at (datetime, server default now)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 89 — Implement POST /predict in backend/app/routes/predict.py

Objective: Implement `POST /predict` in `backend/app/routes/predict.py`: accept multipart image upload, validate file type/size, run preprocessing + inference.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `POST /predict`, `backend/app/routes/predict.py`

Detailed Requirements

- Implement `POST /predict` in `backend/app/routes/predict.py`: accept multipart image upload, validate file type/size, run preprocessing + inference.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Route: POST /predict. Response/request shape should match the corresponding Pydantic schema already defined for this module.

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 90 — Add confidence-threshold logic to /predict that sets is_confident: false and an "uncerta...

Objective: Add confidence-threshold logic to `/predict` that sets `is_confident: false` and an "uncertain result" message below the production threshold.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `/predict`

Detailed Requirements

- Add confidence-threshold logic to `/predict` that sets `is_confident: false` and an "uncertain result" message below the production threshold.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Response/request shape should match the corresponding Pydantic schema already defined for this module.

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 91 — Wire /predict to look up the predicted disease's info from the diseases table and includ...

Objective: Wire `/predict` to look up the predicted disease's info from the `diseases` table and include it in the response.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `/predict`

Detailed Requirements

- Wire `/predict` to look up the predicted disease's info from the `diseases` table and include it in the response.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Disease table columns — id (PK) · crop (str, required) · disease_name (str, required) · description (text) · symptoms (text/array) · prevention (text/array) · management (text/array)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 92 — Wire /predict to save a new row to the predictions table with the correct user_id from...

Objective: Wire `/predict` to save a new row to the `predictions` table with the correct `user_id` from the authenticated request.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `/predict`

Detailed Requirements

- Wire `/predict` to save a new row to the `predictions` table with the correct `user_id` from the authenticated request.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.

- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Response/request shape should match the corresponding Pydantic schema already defined for this module.

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 93 — Add optional image persistence

Objective: Add optional image persistence: upload the raw image to object storage and store its URL on the prediction row.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add optional image persistence: upload the raw image to object storage and store its URL on the prediction row.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Prediction table columns — id (PK) · user_id (FK -> users.id, indexed, required) · image_url (str, nullable) · crop (str, required) · disease (str, required) · confidence (float 0-1, required) · is_confident (bool, required) · created_at (datetime, server default now)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 94 — Add robust error handling for invalid/corrupt image uploads and for model-inference failures

Objective: Add robust error handling for invalid/corrupt image uploads and for model-inference failures.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add robust error handling for invalid/corrupt image uploads and for model-inference failures.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Response/request shape should match the corresponding Pydantic schema already defined for this module.

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 95 — Write backend/tests/test_predict.py covering a valid prediction, a low-confidence predicti...

Objective: Write `backend/tests/test_predict.py` covering a valid prediction, a low-confidence prediction, and an invalid file upload.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `backend/tests/test_predict.py`

Detailed Requirements

- Write `backend/tests/test_predict.py` covering a valid prediction, a low-confidence prediction, and an invalid file upload.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Prediction table columns — id (PK) · user_id (FK -> users.id, indexed, required) · image_url (str, nullable) · crop (str, required) · disease (str, required) · confidence (float 0-1, required) · is_confident (bool, required) · created_at (datetime, server default now)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

2.F Disease Information & History Module

PROMPT 96 — Implement GET /diseases listing all disease records (crop, name, short description) for th...

Objective: Implement `GET /diseases` listing all disease records (crop, name, short description) for the About/Methodology page.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `GET /diseases`

Detailed Requirements

- Implement `GET /diseases` listing all disease records (crop, name, short description) for the About/Methodology page.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Route: GET /diseases. Disease table columns — id (PK) · crop (str, required) · disease_name (str, required) · description (text) · symptoms (text/array) · prevention (text/array) · management (text/array)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 97 — Implement GET /diseases/{id} returning full detail for one disease

Objective: Implement `GET /diseases/{id}` returning full detail for one disease.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `GET /diseases/{id}`

Detailed Requirements

- Implement `GET /diseases/{id}` returning full detail for one disease.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Route: GET /diseases/{id}. Disease table columns — id (PK) · crop (str, required) · disease_name (str, required) · description (text) · symptoms (text/array) · prevention (text/array) · management (text/array)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 98 — Implement GET /predictions/history returning the authenticated user's own predictions, mos...

Objective: Implement `GET /predictions/history` returning the authenticated user's own predictions, most recent first, with pagination support.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `GET /predictions/history`

Detailed Requirements

- Implement `GET /predictions/history` returning the authenticated user's own predictions, most recent first, with pagination support.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Route: GET /predictions/history. User table columns — id (PK, UUID/int) · name (str, required) · email (str, unique, indexed, required) · password_hash (str, required, bcrypt) · phone (str, nullable) · location (str, nullable) · profile_photo_url (str, nullable) · preferred_crops (str/array, nullable) · created_at (datetime, server default now) · last_login_at (datetime, nullable)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 99 — Implement POST /predictions/{id}/feedback letting a user confirm or correct a past predict...

Objective: Implement `POST /predictions/{id}/feedback` letting a user confirm or correct a past prediction, writing to the `feedback` table.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `POST /predictions/{id}/feedback`

Detailed Requirements

- Implement `POST /predictions/{id}/feedback` letting a user confirm or correct a past prediction, writing to the `feedback` table.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Route: POST /predictions/{id}/feedback. Feedback table columns — id (PK) · prediction_id (FK -> predictions.id, required) · user_id (FK -> users.id, required) · actual_disease (str, nullable) · is_correct (bool, required) · comment (text, nullable) · created_at (datetime, server default now)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.

- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

PROMPT 100 — Write backend/tests/test_history_feedback.py covering history scoping (user A cannot see u...

Objective: Write `backend/tests/test_history_feedback.py` covering history scoping (user A cannot see user B's predictions) and feedback submission.

Why This Matters: This endpoint is one of the core data-access points the personalized frontend depends on. It must correctly scope every query to the authenticated user via `get_current_user`, since that is what guarantees one user can never see or alter another user's data.

Primary File(s): `backend/tests/test_history_feedback.py`

Detailed Requirements

- Write `backend/tests/test_history_feedback.py` covering history scoping (user A cannot see user B's predictions) and feedback submission.
- Protect the route with the `get_current_user` dependency; never trust a user ID supplied in the request body or URL for authorization decisions.
- Validate and parse the request using an explicit Pydantic schema, not raw dict access.
- Return a response matching exactly the shape already documented in the backend API table (field names and types).
- Use appropriate HTTP status codes (200/201 success, 400 validation, 401 unauthenticated, 404 not found, 422 unprocessable).
- Write at least one automated test for this route covering both the happy path and one failure path.

Technical Specification

Feedback table columns — id (PK) · prediction_id (FK -> predictions.id, required) · user_id (FK -> users.id, required) · actual_disease (str, nullable) · is_correct (bool, required) · comment (text, nullable) · created_at (datetime, server default now)

Acceptance Criteria

- ☐ Calling the route with a valid JWT and valid input returns the documented response shape with a 2xx status.
- ☐ Calling the route without a token returns 401.
- ☐ Calling the route for one user never returns or mutates another user's rows (verified with two distinct test accounts).

Edge Cases & Notes

- Decide and implement sensible pagination defaults (e.g. limit/offset or cursor) if the route returns a list that could grow large.
- Handle the “no data yet” case (e.g. a brand-new user with zero predictions) with an empty array/sensible default, not an error.

2.G Middleware, Error Handling, Testing & Docs

PROMPT 101 — Add global exception handlers in backend/app/main.py for validation errors and unhandled s...

Objective: Add global exception handlers in `backend/app/main.py` for validation errors and unhandled server errors, returning consistent JSON error shapes.

Why This Matters: These cross-cutting concerns (error handling, logging, CORS, testing infrastructure, docs) determine whether the backend behaves predictably and debuggably once real traffic — or real bugs — start hitting it, rather than only working in the happy-path demo.

Primary File(s): `backend/app/main.py`

Detailed Requirements

- Add global exception handlers in `backend/app/main.py` for validation errors and unhandled server errors, returning consistent JSON error shapes.
- Centralize the behaviour (exception handler, middleware, fixture) in one place rather than duplicating logic per-route.
- Ensure error responses returned to the client never leak internal details (stack traces, raw exception messages, SQL).
- Keep this infrastructure code covered by at least one automated test confirming it behaves as expected under a failure condition.
- Document the behaviour added here in the relevant README/docs file so it is discoverable by the rest of the team.

Technical Specification

Applies at the FastAPI application level (`backend/app/main.py`) or within `backend/tests/` as appropriate — this is shared infrastructure used by every route module, not a single endpoint.

Acceptance Criteria

- ☐ The behaviour described in the objective is demonstrably active (e.g. an intentionally-triggered error returns the expected consistent JSON error shape).
- ☐ No existing test in the suite regresses as a result of this change.
- ☐ The change is documented where a new team member would look for it.

Edge Cases & Notes

- Confirm the change behaves correctly both in local development and under the production environment configuration.

PROMPT 102 — Add request-logging middleware to backend/app/main.py for debugging and audit purposes

Objective: Add request-logging middleware to `backend/app/main.py` for debugging and audit purposes.

Why This Matters: These cross-cutting concerns (error handling, logging, CORS, testing infrastructure, docs) determine whether the backend behaves predictably and debuggably once real traffic — or real bugs — start hitting it, rather than only working in the happy-path demo.

Primary File(s): `backend/app/main.py`

Detailed Requirements

- Add request-logging middleware to `backend/app/main.py` for debugging and audit purposes.

- Centralize the behaviour (exception handler, middleware, fixture) in one place rather than duplicating logic per-route.
- Ensure error responses returned to the client never leak internal details (stack traces, raw exception messages, SQL).
- Keep this infrastructure code covered by at least one automated test confirming it behaves as expected under a failure condition.
- Document the behaviour added here in the relevant README/docs file so it is discoverable by the rest of the team.

Technical Specification

Applies at the FastAPI application level (`backend/app/main.py`) or within `backend/tests/` as appropriate — this is shared infrastructure used by every route module, not a single endpoint.

Acceptance Criteria

- ☐ The behaviour described in the objective is demonstrably active (e.g. an intentionally-triggered error returns the expected consistent JSON error shape).
- ☐ No existing test in the suite regresses as a result of this change.
- ☐ The change is documented where a new team member would look for it.

Edge Cases & Notes

- Confirm the change behaves correctly both in local development and under the production environment configuration.

PROMPT 103 — Configure CORS explicitly to allow only the deployed frontend origin(s)

Objective: Configure CORS explicitly to allow only the deployed frontend origin(s).

Why This Matters: These cross-cutting concerns (error handling, logging, CORS, testing infrastructure, docs) determine whether the backend behaves predictably and debuggably once real traffic — or real bugs — start hitting it, rather than only working in the happy-path demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Configure CORS explicitly to allow only the deployed frontend origin(s).
- Centralize the behaviour (exception handler, middleware, fixture) in one place rather than duplicating logic per-route.
- Ensure error responses returned to the client never leak internal details (stack traces, raw exception messages, SQL).
- Keep this infrastructure code covered by at least one automated test confirming it behaves as expected under a failure condition.
- Document the behaviour added here in the relevant README/docs file so it is discoverable by the rest of the team.

Technical Specification

Applies at the FastAPI application level (`backend/app/main.py`) or within `backend/tests/` as appropriate — this is shared infrastructure used by every route module, not a single endpoint.

Acceptance Criteria

- ☐ The behaviour described in the objective is demonstrably active (e.g. an intentionally-triggered error returns the expected consistent JSON error shape).
- ☐ No existing test in the suite regresses as a result of this change.

- ☐ The change is documented where a new team member would look for it.

Edge Cases & Notes

- Confirm the change behaves correctly both in local development and under the production environment configuration.

PROMPT 104 — Add input-size limits (max upload size) to the FastAPI app configuration

Objective: Add input-size limits (max upload size) to the FastAPI app configuration.

Why This Matters: These cross-cutting concerns (error handling, logging, CORS, testing infrastructure, docs) determine whether the backend behaves predictably and debuggably once real traffic — or real bugs — start hitting it, rather than only working in the happy-path demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add input-size limits (max upload size) to the FastAPI app configuration.
- Centralize the behaviour (exception handler, middleware, fixture) in one place rather than duplicating logic per-route.
- Ensure error responses returned to the client never leak internal details (stack traces, raw exception messages, SQL).
- Keep this infrastructure code covered by at least one automated test confirming it behaves as expected under a failure condition.
- Document the behaviour added here in the relevant README/docs file so it is discoverable by the rest of the team.

Technical Specification

Applies at the FastAPI application level (`backend/app/main.py`) or within `backend/tests/` as appropriate — this is shared infrastructure used by every route module, not a single endpoint.

Acceptance Criteria

- ☐ The behaviour described in the objective is demonstrably active (e.g. an intentionally-triggered error returns the expected consistent JSON error shape).
- ☐ No existing test in the suite regresses as a result of this change.
- ☐ The change is documented where a new team member would look for it.

Edge Cases & Notes

- Confirm the change behaves correctly both in local development and under the production environment configuration.

PROMPT 105 — Write backend/tests/conftest.py setting up a test database and a reusable authenticated te...

Objective: Write `backend/tests/conftest.py` setting up a test database and a reusable authenticated test-client fixture.

Why This Matters: These cross-cutting concerns (error handling, logging, CORS, testing infrastructure, docs) determine whether the backend behaves predictably and debuggably once real traffic — or real bugs — start hitting it, rather than only working in the happy-path demo.

Primary File(s): `backend/tests/conftest.py`

Detailed Requirements

- Write `backend/tests/conftest.py` setting up a test database and a reusable authenticated test-client fixture.
- Centralize the behaviour (exception handler, middleware, fixture) in one place rather than duplicating logic per-route.
- Ensure error responses returned to the client never leak internal details (stack traces, raw exception messages, SQL).
- Keep this infrastructure code covered by at least one automated test confirming it behaves as expected under a failure condition.
- Document the behaviour added here in the relevant README/docs file so it is discoverable by the rest of the team.

Technical Specification

Applies at the FastAPI application level (`backend/app/main.py`) or within `backend/tests/` as appropriate — this is shared infrastructure used by every route module, not a single endpoint.

Acceptance Criteria

- ☐ The behaviour described in the objective is demonstrably active (e.g. an intentionally-triggered error returns the expected consistent JSON error shape).
- ☐ No existing test in the suite regresses as a result of this change.
- ☐ The change is documented where a new team member would look for it.

Edge Cases & Notes

- Confirm the change behaves correctly both in local development and under the production environment configuration.

PROMPT 106 — Achieve and verify test coverage across auth, users, predict, history, and feedback modules;...

Objective: Achieve and verify test coverage across auth, users, predict, history, and feedback modules; document results in `backend/TESTING.md`.

Why This Matters: These cross-cutting concerns (error handling, logging, CORS, testing infrastructure, docs) determine whether the backend behaves predictably and debuggably once real traffic — or real bugs — start hitting it, rather than only working in the happy-path demo.

Primary File(s): `backend/TESTING.md`

Detailed Requirements

- Achieve and verify test coverage across auth, users, predict, history, and feedback modules; document results in `backend/TESTING.md`.
- Centralize the behaviour (exception handler, middleware, fixture) in one place rather than duplicating logic per-route.
- Ensure error responses returned to the client never leak internal details (stack traces, raw exception messages, SQL).
- Keep this infrastructure code covered by at least one automated test confirming it behaves as expected under a failure condition.
- Document the behaviour added here in the relevant README/docs file so it is discoverable by the rest of the team.

Technical Specification

Applies at the FastAPI application level (`backend/app/main.py`) or within `backend/tests/` as appropriate — this is shared infrastructure used by every route module, not a single endpoint.

Acceptance Criteria

- ☐ The behaviour described in the objective is demonstrably active (e.g. an intentionally-triggered error returns the expected consistent JSON error shape).
- ☐ No existing test in the suite regresses as a result of this change.
- ☐ The change is documented where a new team member would look for it.

Edge Cases & Notes

- Confirm the change behaves correctly both in local development and under the production environment configuration.

PROMPT 107 — Review FastAPI's automatic OpenAPI/Swagger docs at /docs; annotate routes with clear summa...

Objective: Review FastAPI's automatic OpenAPI/Swagger docs at `/docs`; annotate routes with clear summaries and example responses.

Why This Matters: These cross-cutting concerns (error handling, logging, CORS, testing infrastructure, docs) determine whether the backend behaves predictably and debuggably once real traffic — or real bugs — start hitting it, rather than only working in the happy-path demo.

Primary File(s): `/docs`

Detailed Requirements

- Review FastAPI's automatic OpenAPI/Swagger docs at `/docs`; annotate routes with clear summaries and example responses.
- Centralize the behaviour (exception handler, middleware, fixture) in one place rather than duplicating logic per-route.
- Ensure error responses returned to the client never leak internal details (stack traces, raw exception messages, SQL).
- Keep this infrastructure code covered by at least one automated test confirming it behaves as expected under a failure condition.
- Document the behaviour added here in the relevant README/docs file so it is discoverable by the rest of the team.

Technical Specification

Applies at the FastAPI application level (`backend/app/main.py`) or within `backend/tests/` as appropriate — this is shared infrastructure used by every route module, not a single endpoint.

Acceptance Criteria

- ☐ The behaviour described in the objective is demonstrably active (e.g. an intentionally-triggered error returns the expected consistent JSON error shape).
- ☐ No existing test in the suite regresses as a result of this change.
- ☐ The change is documented where a new team member would look for it.

Edge Cases & Notes

- Confirm the change behaves correctly both in local development and under the production environment configuration.

PROMPT 108 — Write backend/README.md documenting setup, environment variables, running locally, running...

Objective: Write `backend/README.md` documenting setup, environment variables, running locally, running tests, and the full API reference.

Why This Matters: These cross-cutting concerns (error handling, logging, CORS, testing infrastructure, docs) determine whether the backend behaves predictably and debuggably once real traffic — or real bugs — start hitting it, rather than only working in the happy-path demo.

Primary File(s): `backend/README.md`

Detailed Requirements

- Write `backend/README.md` documenting setup, environment variables, running locally, running tests, and the full API reference.
- Centralize the behaviour (exception handler, middleware, fixture) in one place rather than duplicating logic per-route.
- Ensure error responses returned to the client never leak internal details (stack traces, raw exception messages, SQL).
- Keep this infrastructure code covered by at least one automated test confirming it behaves as expected under a failure condition.
- Document the behaviour added here in the relevant README/docs file so it is discoverable by the rest of the team.

Technical Specification

Applies at the FastAPI application level (`backend/app/main.py`) or within `backend/tests/` as appropriate — this is shared infrastructure used by every route module, not a single endpoint.

Acceptance Criteria

- ☐ The behaviour described in the objective is demonstrably active (e.g. an intentionally-triggered error returns the expected consistent JSON error shape).
- ☐ No existing test in the suite regresses as a result of this change.
- ☐ The change is documented where a new team member would look for it.

Edge Cases & Notes

- Confirm the change behaves correctly both in local development and under the production environment configuration.

PROMPT 109 — Write a curl/HTTP request reference covering every endpoint, saved to backend/docs/api-requ...

Objective: Write a curl/HTTP request reference covering every endpoint, saved to `backend/docs/api-requests.http`.

Why This Matters: These cross-cutting concerns (error handling, logging, CORS, testing infrastructure, docs) determine whether the backend behaves predictably and debuggably once real traffic — or real bugs — start hitting it, rather than only working in the happy-path demo.

Primary File(s): `backend/docs/api-requests.http`

Detailed Requirements

- Write a curl/HTTP request reference covering every endpoint, saved to `backend/docs/api-requests.http`.

- Centralize the behaviour (exception handler, middleware, fixture) in one place rather than duplicating logic per-route.
- Ensure error responses returned to the client never leak internal details (stack traces, raw exception messages, SQL).
- Keep this infrastructure code covered by at least one automated test confirming it behaves as expected under a failure condition.
- Document the behaviour added here in the relevant README/docs file so it is discoverable by the rest of the team.

Technical Specification

Applies at the FastAPI application level (`backend/app/main.py`) or within `backend/tests/` as appropriate — this is shared infrastructure used by every route module, not a single endpoint.

Acceptance Criteria

- ☐ The behaviour described in the objective is demonstrably active (e.g. an intentionally-triggered error returns the expected consistent JSON error shape).
- ☐ No existing test in the suite regresses as a result of this change.
- ☐ The change is documented where a new team member would look for it.

Edge Cases & Notes

- Confirm the change behaves correctly both in local development and under the production environment configuration.

PROMPT 110 — Do a final backend code-review pass

Objective: Do a final backend code-review pass: consistent naming, remove dead code, confirm every protected route uses `get_current_user`.

Why This Matters: These cross-cutting concerns (error handling, logging, CORS, testing infrastructure, docs) determine whether the backend behaves predictably and debuggably once real traffic — or real bugs — start hitting it, rather than only working in the happy-path demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Do a final backend code-review pass: consistent naming, remove dead code, confirm every protected route uses `get_current_user`.
- Centralize the behaviour (exception handler, middleware, fixture) in one place rather than duplicating logic per-route.
- Ensure error responses returned to the client never leak internal details (stack traces, raw exception messages, SQL).
- Keep this infrastructure code covered by at least one automated test confirming it behaves as expected under a failure condition.
- Document the behaviour added here in the relevant README/docs file so it is discoverable by the rest of the team.

Technical Specification

Applies at the FastAPI application level (`backend/app/main.py`) or within `backend/tests/` as appropriate — this is shared infrastructure used by every route module, not a single endpoint.

Acceptance Criteria

- ☐ The behaviour described in the objective is demonstrably active (e.g. an intentionally-triggered error returns the expected consistent JSON error shape).
- ☐ No existing test in the suite regresses as a result of this change.
- ☐ The change is documented where a new team member would look for it.

Edge Cases & Notes

- Confirm the change behaves correctly both in local development and under the production environment configuration.
